



영화 관객수 예측 경진대회

미니 프로젝트(3)

YB 1조 김재경, 유재은, 이우림, 전예원

목차

1. 데이터 및 라이브러리 로딩

2. EDA 및 데이터 전처리

- 1) title
- 2) release_time
- 3) time
- 4) distributor
- 5) dir_prev_num
- 6) genre
- 7) dir_prev_bfnum
- 8) 그 외 변수들
- 9) 문자 칼럼 제거
- 10) 결측치 처리

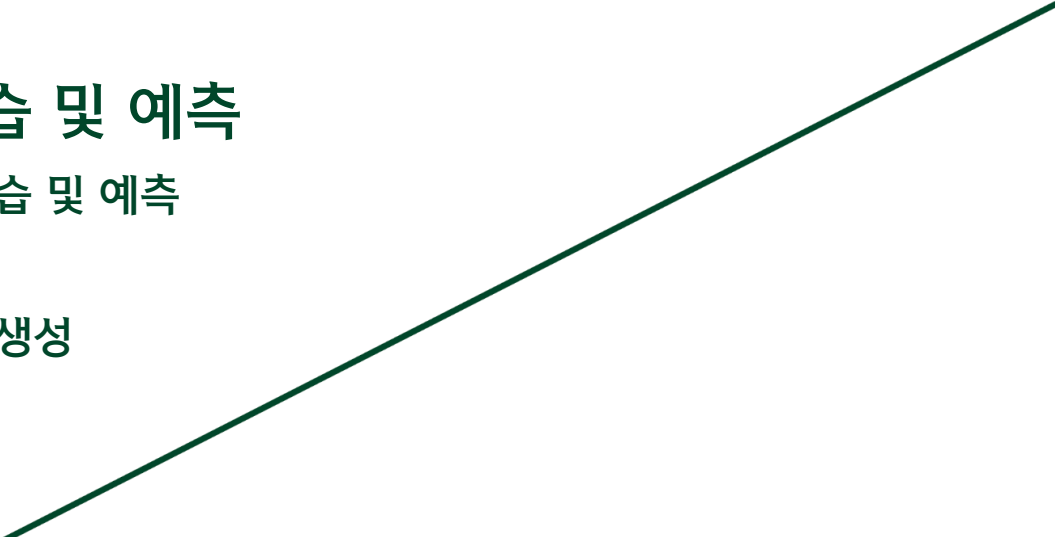
3. 스케일링

- 1) 데이터 분리
- 2) 스케일링

4. 모델 생성

- 1) 회귀 모델 정의
- 2) 스택킹

5. 모델 학습 및 예측

- 1) 개별 모델 학습 및 예측
 - 2) 앙상블
 - 3) 제출 데이터 생성
- 

1. 데이터 및 라이브러리 로딩

```
import pandas as pd
import numpy as np
import warnings
warnings.filterwarnings('ignore')

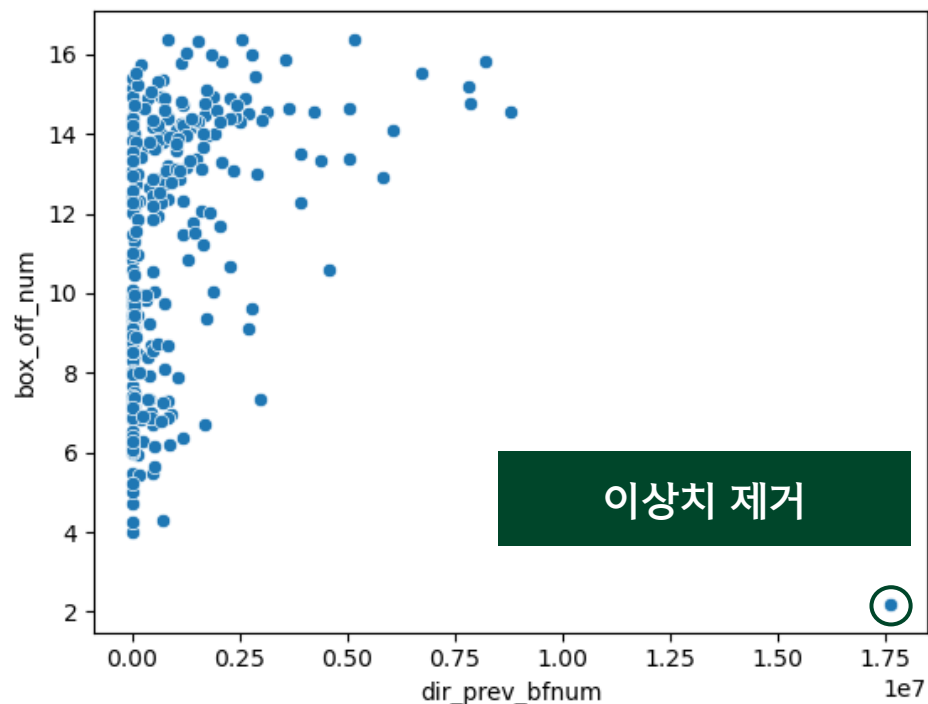
import seaborn as sns
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import KFold, train_test_split
from sklearn.preprocessing import StandardScaler, RobustScaler
from sklearn.linear_model import Lasso, ElasticNet
from sklearn.kernel_ridge import KernelRidge
from sklearn.ensemble import ExtraTreesRegressor
from xgboost import XGBRegressor
import lightgbm as lgb
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.base import clone

from google.colab import drive
drive.mount('/content/drive')
train=pd.read_csv('/content/drive/MyDrive/movies_train.csv')
test=pd.read_csv('/content/drive/MyDrive/movies_test.csv')
submission=pd.read_csv('/content/drive/MyDrive/submission.csv')
```

- 필요한 라이브러리 импорт
- train: 학습 데이터
- test: 예측 데이터
- submission: 제출 양식

1. 데이터 및 라이브러리 로딩

```
sns.scatterplot(x=train['dir_prev_bfnum'], y=np.log1p(train['box_off_num']))
```



```
print(train[train['dir_prev_bfnum']>17500000].index)
```

```
Index([338], dtype='int64')
```

```
train.loc[338]
```

338	
title	타부 : 금지된 사랑
distributor	고구마공작소
genre	멜로/로맨스
release_time	2015-09-03
time	78
screening_rat	청소년 관람불가
director	고구마
dir_prev_bfnum	17615314.0
dir_prev_num	1
num_staff	27
num_actor	3
box_off_num	8

```
dtype: object
```

```
train = train.drop(index=338).reset_index(drop=True)
```

1. 데이터 및 라이브러리 로딩

```
train.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 599 entries, 0 to 598
```

```
Data columns (total 12 columns):
```

#	Column	Non-Null Count	Dtype
0	title	599 non-null	object
1	distributor	599 non-null	object
2	genre	599 non-null	object
3	release_time	599 non-null	object
4	time	599 non-null	int64
5	screening_rat	599 non-null	object
6	director	599 non-null	object
7	dir_prev_bfnum	269 non-null	float64
8	dir_prev_num	599 non-null	int64
9	num_staff	599 non-null	int64
10	num_actor	599 non-null	int64
11	box_off_num	599 non-null	int64

```
dtypes: float64(1), int64(5), object(6)
```

```
memory usage: 56.3+ KB
```

- title : 영화의 제목
- distributor : 배급사
- genre : 장르
- release_time : 개봉일
- time : 상영시간(분)
- screening_rat : 상영등급
- director : 감독이름
- dir_prev_bfnum : 이 영화를 만들기 전 참여한 영화에서의 평균 관객수
- dir_prev_num : 이 영화를 만들기 전 참여한 영화의 개수
- num_staff : 스텝수
- num_actor : 주연배우수
- box_off_num : 관객수

1. 데이터 및 라이브러리 로딩

- box_off_num 분리
- box_off_num 로그 변환
- train, test 데이터 합치기

```
train_x=train.drop('box_off_num',axis=1)
train_y=train.box_off_num
train_y_log = np.log1p(train_y)
n1=len(train_x)# 이상치 하나 제거!!!!
df=pd.concat([train_x,test],ignore_index=True) #train test 합쳐서 전처리 후 분리
```

2-1. title

- title은 분석에 필요하지 않으므로 제거

```
df.drop('title',axis=1,inplace=True)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 842 entries, 0 to 841  
Data columns (total 10 columns):  
#   Column                Non-Null Count  Dtype  
---  ---  
0   distributor            842 non-null    object  
1   genre                  842 non-null    object  
2   release_time           842 non-null    object  
3   time                   842 non-null    int64  
4   screening_rat          842 non-null    object  
5   director               842 non-null    object  
6   dir_prev_bfnum         376 non-null    float64  
7   dir_prev_num           842 non-null    int64  
8   num_staff              842 non-null    int64  
9   num_actor              842 non-null    int64  
dtypes: float64(1), int64(4), object(5)  
memory usage: 65.9+ KB
```

2-2. release_time

- datetime으로 변환
- 'year', 'month', 'weekday', 'is_weekend' 변수 생성

```
train['release_dt'] = pd.to_datetime(train['release_time'])
test['release_dt'] = pd.to_datetime(test['release_time'])

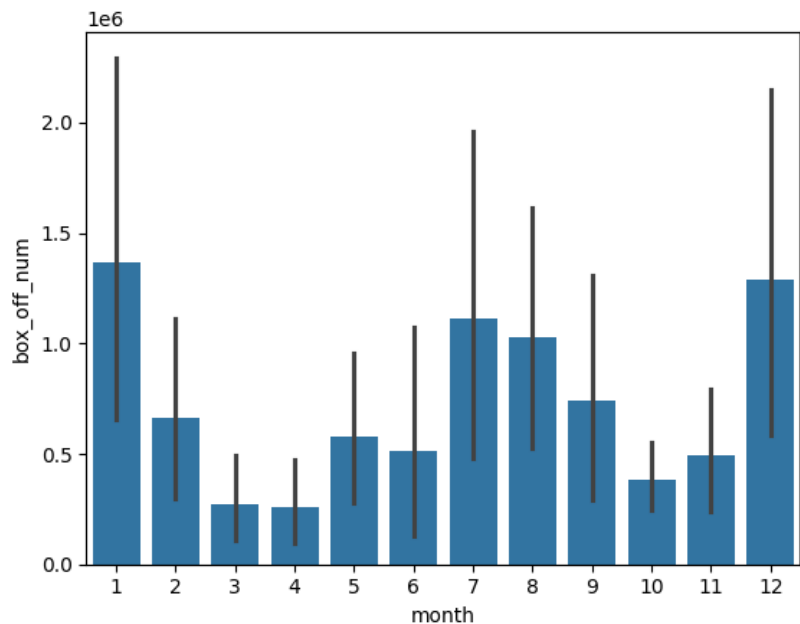
train['year'] = train['release_dt'].dt.year
train['month'] = train['release_dt'].dt.month
train['weekday'] = train['release_dt'].dt.weekday
train['is_weekend'] = train['weekday'].isin([4, 5, 6]).astype(int)

test['year'] = test['release_dt'].dt.year
test['month'] = test['release_dt'].dt.month
test['weekday'] = test['release_dt'].dt.weekday
test['is_weekend'] = test['weekday'].isin([4, 5, 6]).astype(int)
```

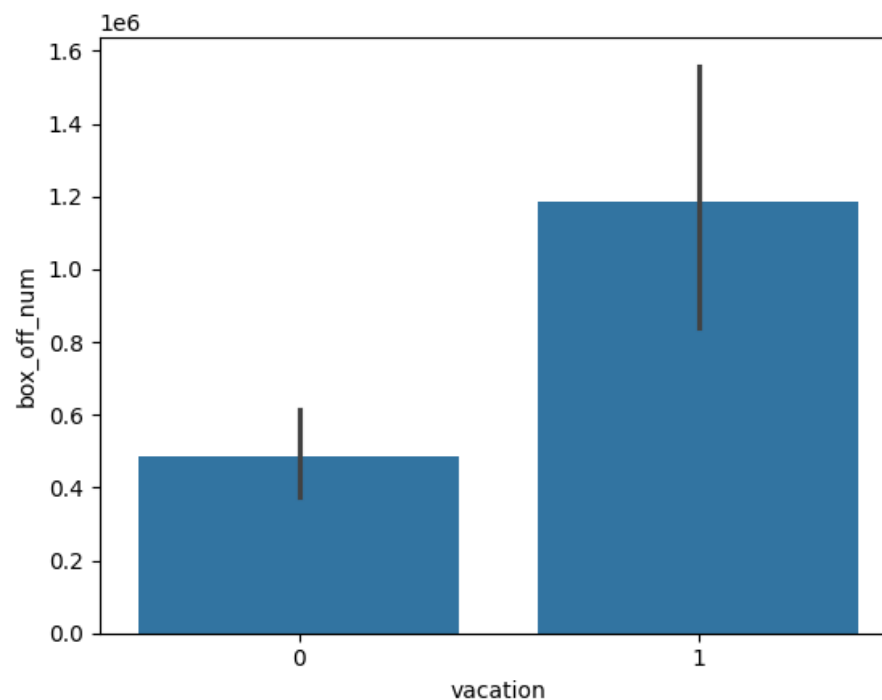

2-2. release_time

- month의 경우 방학인 달에서 box_off_num이 높게 나타남
- 1, 7, 8, 12월이 1, 나머지가 0인 vacation 변수로 할당

```
# vacation (기존 정의 유지)
sns.barplot(x=train['month'], y=train['box_off_num'], data=df)
train['vacation'] = train['month'].isin([1, 7, 8, 12]).astype(int)
test['vacation'] = test['month'].isin([1, 7, 8, 12]).astype(int)
```



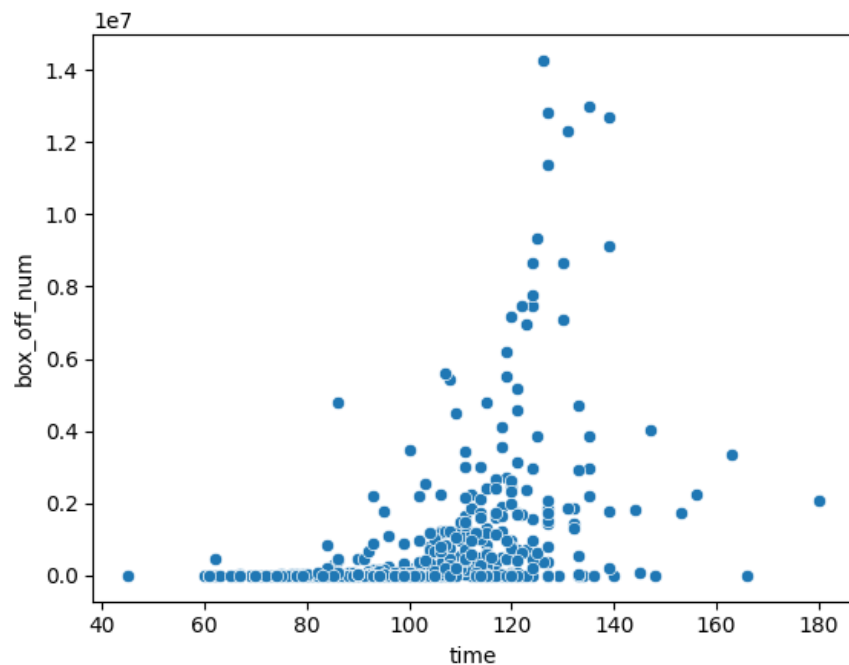
```
sns.barplot(x=train['vacation'], y=train['box_off_num'])
```



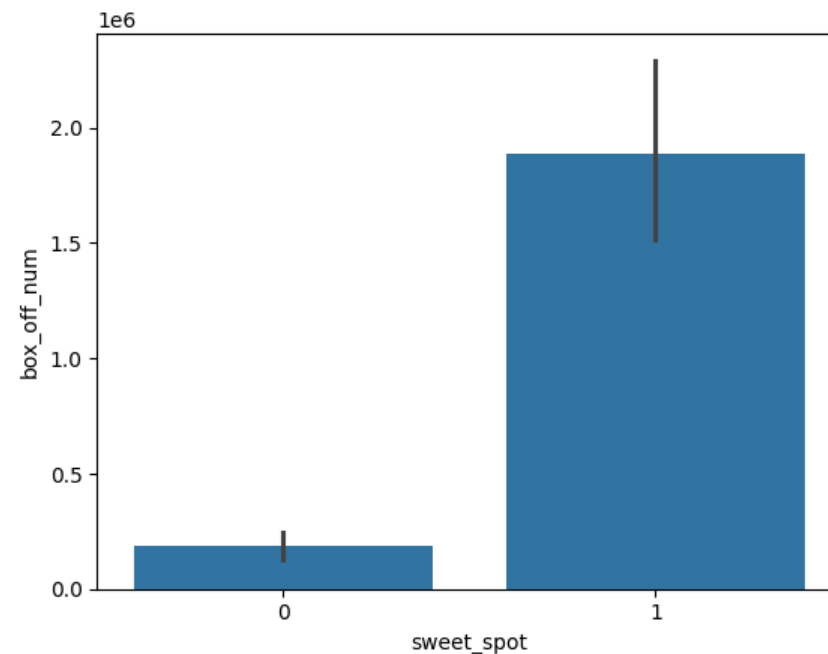
2-3. time

- 상영 시간이 110에서 150분 사이인 경우 관람객이 많음
- sweet_spot이라는 변수로 할당
 - 상영 시간이 110에서 150 사이이면 1, 아니면 0

```
sns.scatterplot(x=train['time'],y=train['box_off_num'])  
# sweet spot (상영시간 110~150분)  
train['sweet_spot'] = train['time'].between(110, 150).astype(int)  
test['sweet_spot'] = test['time'].between(110, 150).astype(int)
```



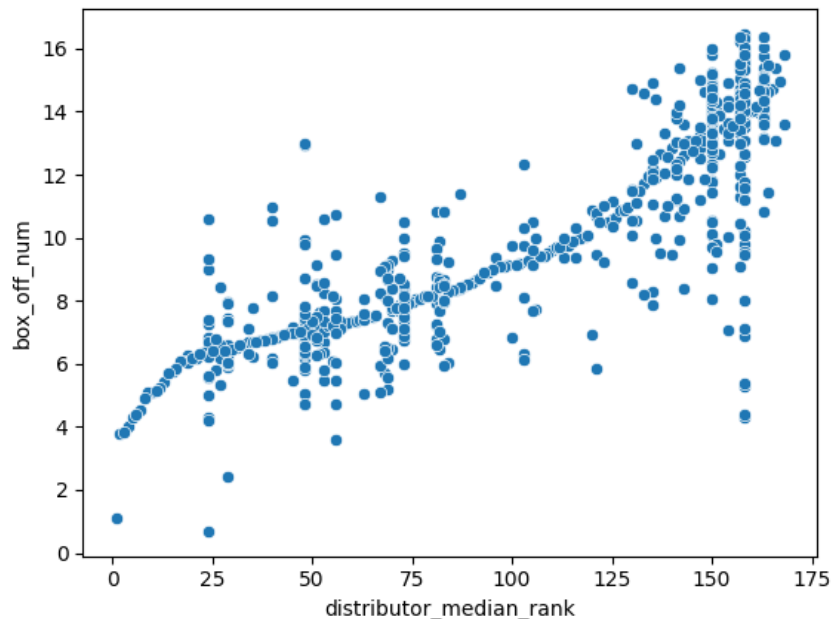
```
sns.barplot(x=train['sweet_spot'],y=train['box_off_num'])
```



2-4. distributor

- 배급사별로 모아서 관객 수의 중앙값 계산
- 배급사별 관객 수의 중앙값을 기준으로 순위 부여
- 'distributor_median_rank'라는 새로운 변수 생성
- test 데이터에 동일한 기준 적용
- test 데이터에 처음 보는 배급사가 나오면, 에러가 나지 않게 '평균 등수'로 빈칸을 채움

```
dist_rank = train.groupby('distributor')['box_off_num'].median().rank()  
train['distributor_median_rank'] = train['distributor'].map(dist_rank)  
test['distributor_median_rank'] = test['distributor'].map(dist_rank).fillna(dist_rank.mean())  
sns.scatterplot(x=train['distributor_median_rank'], y=np.log1p(train['box_off_num']))
```



2-5. dir_prev_num

- 감독이 신인이라면 새로 만든 new_director 변수에서 0을 할당하고, 신인이 아니라면 1을 할당

```
train['new_director'] = (train['dir_prev_num'] == 0).astype(int)
test['new_director'] = (test['dir_prev_num'] == 0).astype(int)
test['new_director']
```

new_director	
0	0
1	0
2	0
3	0
4	0
...	...
238	0
239	1
240	1
241	0
242	1

243 rows × 1 columns

dtype: int64

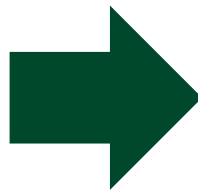
2-6. genre

- 장르별로 평균 관객 수를 계산하고, 평균 관객 수가 많은 순서대로 등수를 매겨 'genre_rank' 변수를 새로 생성함
- test 데이터에 처음 나오는 장르가 있다면, 평균 등수로 채워줌

```
genre_rank = train.groupby('genre')['box_off_num'].mean().rank()
train['genre_rank'] = train['genre'].map(genre_rank)
test['genre_rank'] = test['genre'].map(genre_rank).fillna(genre_rank.mean())
```

```
train['genre'].head(5)
```

	genre
0	액션
1	느와르
2	액션
3	코미디
4	코미디

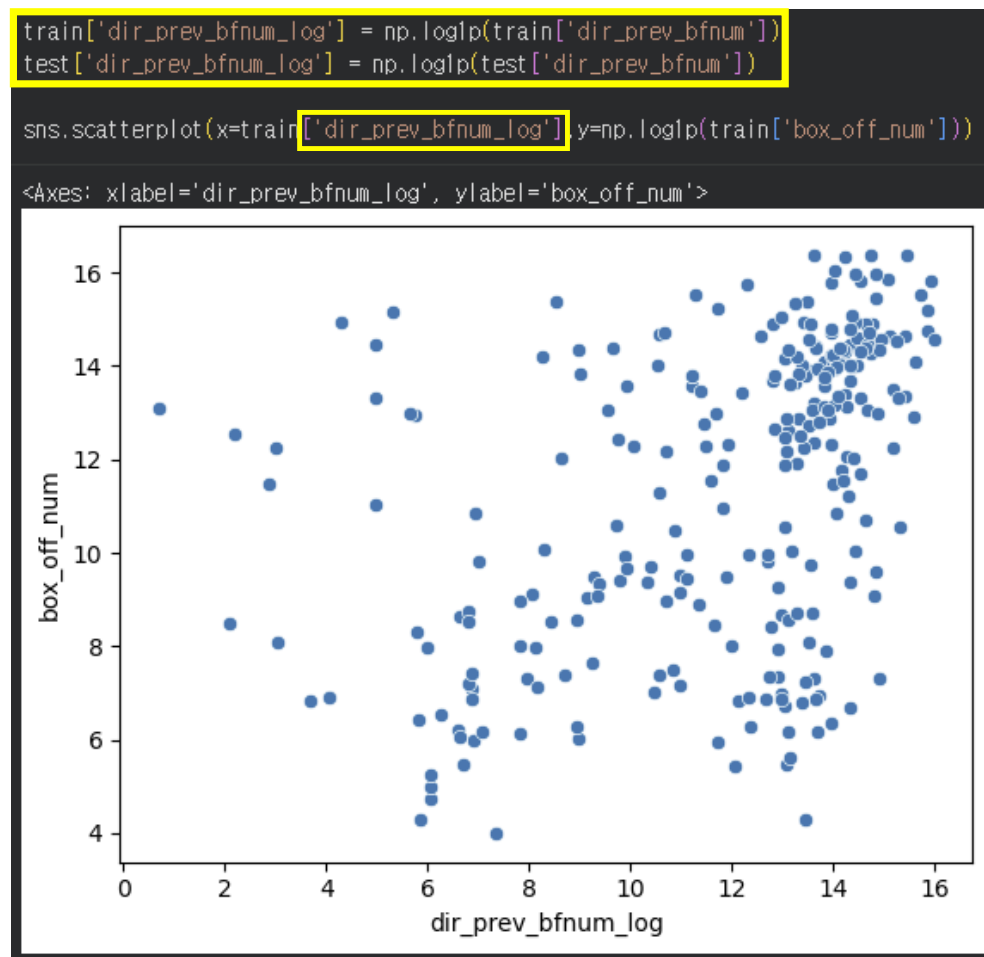
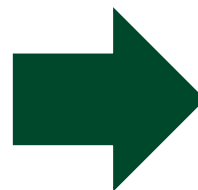
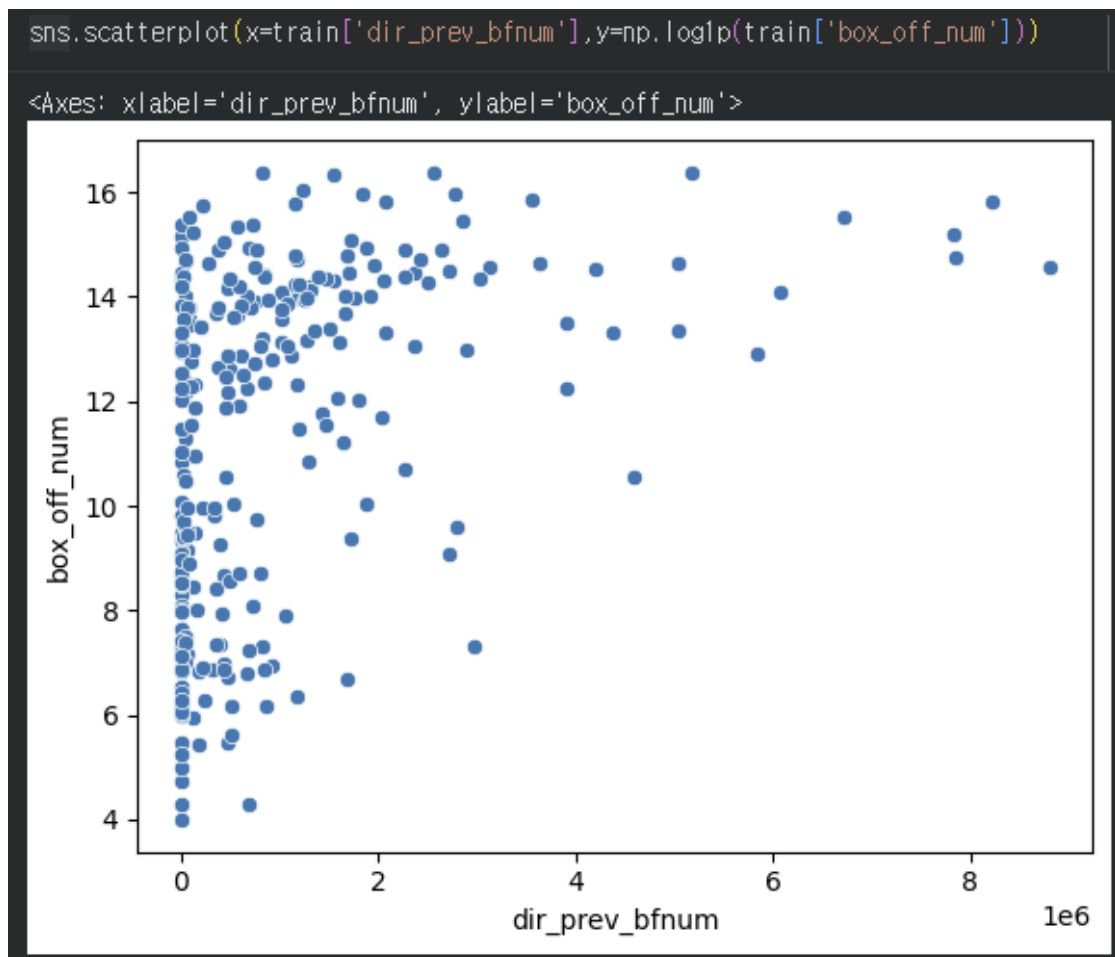


```
train['genre_rank'].head(5)
```

	genre_rank
0	11.0
1	12.0
2	11.0
3	9.0
4	9.0

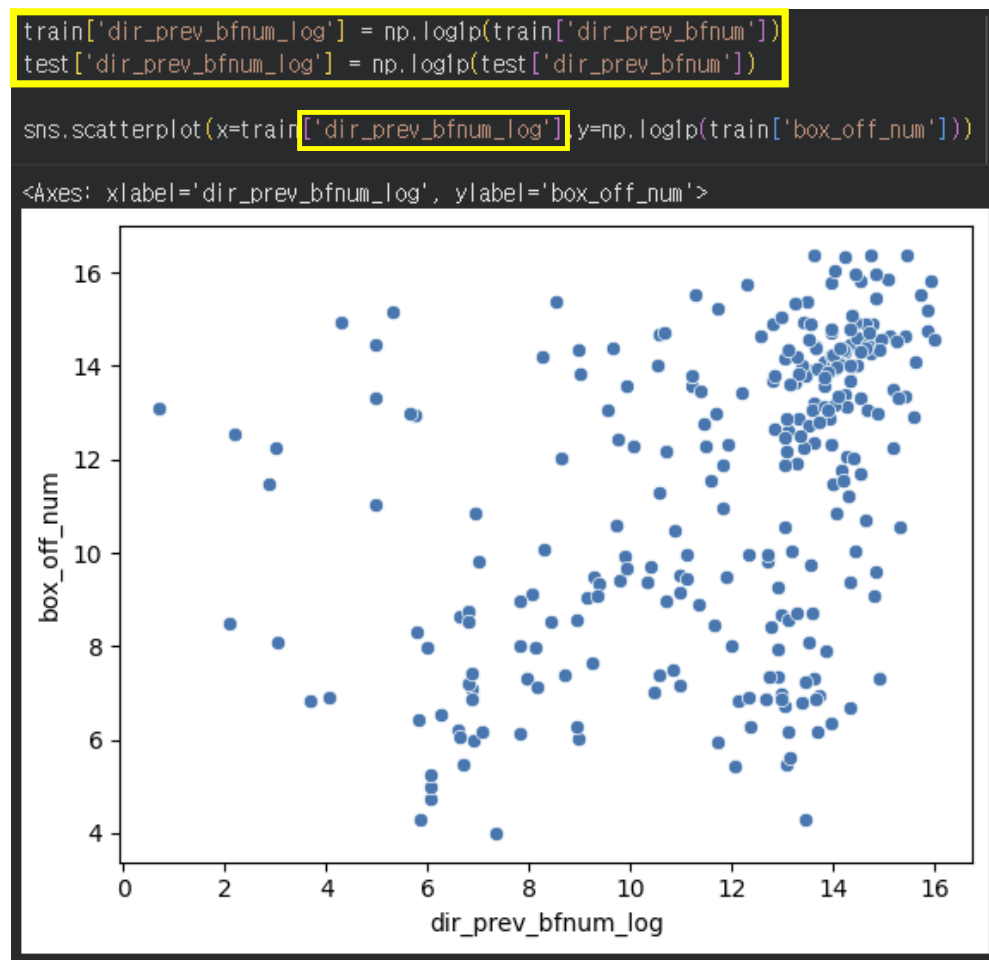
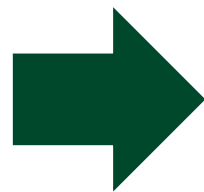
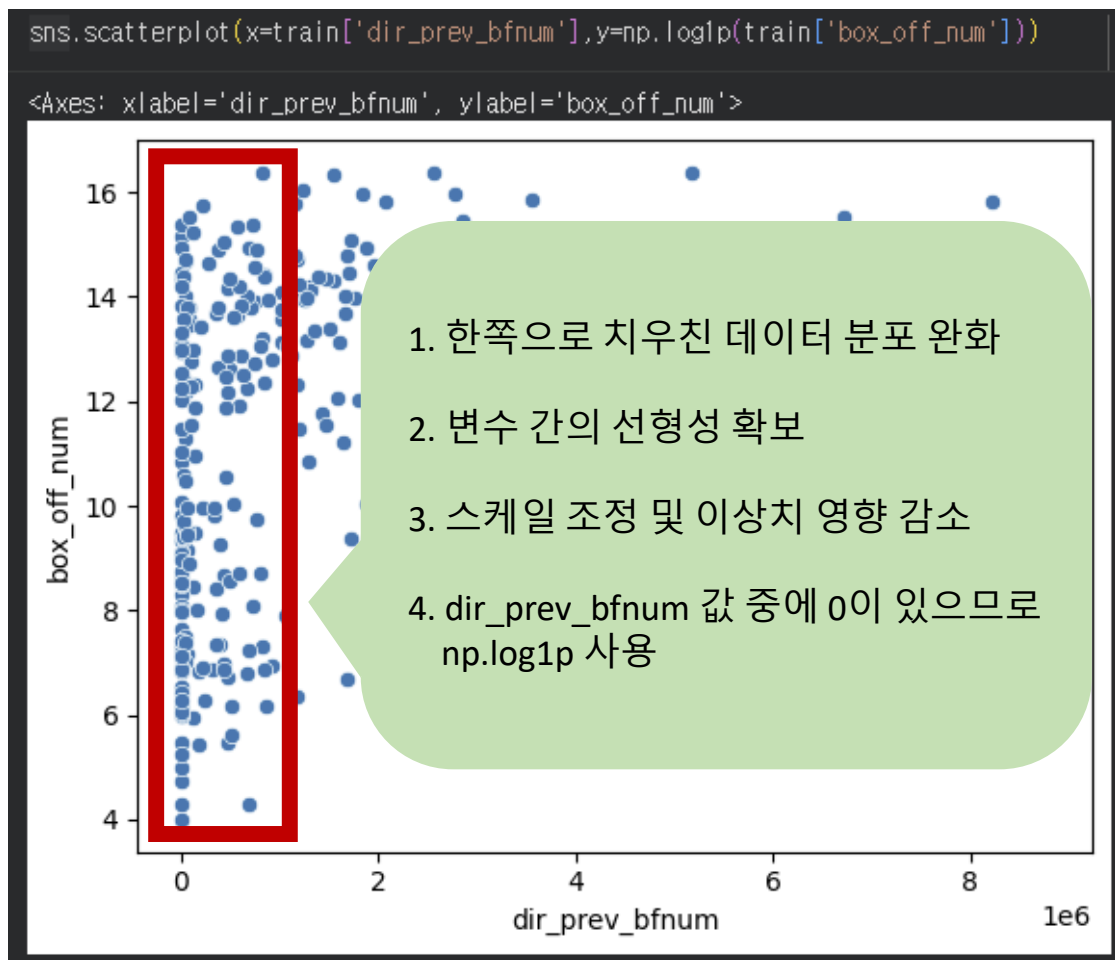
2-7. dir_prev_bfnum

- dir_prev_bfnum에 씌운 값인 dir_prev_bfnum_log 변수를 새로 생성함



2-7. dir_prev_bfnum

- dir_prev_bfnum에 np.log1p를 적용하여 dir_prev_bfnum_log 변수를 새로 생성함



2-8. 그 외 변수들

- 'time', 'dir_prev_num', 'num_staff', 'num_actor', 'distributor_median_rank' 에 np.log1p를 적용하여 'time_log', 'dir_prev_num_log', 'num_staff_log', 'num_actor_log', 'distributor_median_rank_log'를 생성함

```
for col in ['time', 'dir_prev_num', 'num_staff', 'num_actor', 'distributor_median_rank']:
    train[col + '_log'] = np.log1p(train[col])
    test[col + '_log'] = np.log1p(test[col])
```

- 로그 변환 전의 원래 변수들을 제거함

```
log_remove_cols = ['dir_prev_bfnum', 'time', 'dir_prev_num', 'num_staff', 'num_actor', 'distributor_median_rank']
train.drop(columns=log_remove_cols, inplace=True)
test.drop(columns=log_remove_cols, inplace=True)
```


2-9. 문자 컬럼 제거

- 숫자로 변환한 뒤, 더 이상 필요 없어진 문자 컬럼을 제거함

```
obj_cols = train.select_dtypes(include=[object]).columns  
print("문자 컬럼 제거됨:", obj_cols.tolist())
```

```
train.drop(columns=obj_cols, inplace=True)  
test.drop(columns=obj_cols, inplace=True)
```

```
문자 컬럼 제거됨: ['title', 'distributor', 'genre', 'release_time', 'screening_rat', 'director']
```

2-10. 결측치 처리

- 데이터 누수를 막고, 모델 평가의 공정성을 지키기 위해 숫자형 변수들의 결측치를 학습 데이터의 평균으로 전처리해줌
- 테스트 데이터의 결측치도 학습 데이터의 평균 값으로 채운다는 점을 유의해야 함

```
num_cols_train = train.select_dtypes(include=[np.number]).columns
train[num_cols_train] = train[num_cols_train].fillna(train[num_cols_train].mean())

num_cols_test = test.select_dtypes(include=[np.number]).columns
test[num_cols_test] = test[num_cols_test].fillna(train[num_cols_train].mean())
```

3-1. 데이터 분리

타겟(Target) 변수 분리

box_off_num(관객 수)을 y로 저장하여 예측 대상 명확화

입력(Feature) 데이터 구성

box_off_num을 제외한 모든 컬럼을 X로 분리해 모델 학습용 입력으로 사용

```
y = train['box_off_num']  
X = train.drop(['box_off_num'], axis=1)
```

3-2. 스케일링

[데이터 스케일링]

RobustScaler 사용

→ Median & IQR 기반 스케일링

→ 이상치의 영향을 최소화

Train 데이터로만 fit, Test는 동일 기준으로 transform 적용

[Target 변환]

$\log_{10}(y)$ 적용하여 타깃 분포 정규성 개선

큰 값의 스케일 차이를 줄여 모델 성능 개선 기대

```
scaler = RobustScaler()  
X_scaled = scaler.fit_transform(X)  
test_scaled = scaler.transform(test)  
  
train_y_log = np.log1p(y)
```

4-1. 회귀 모델 정의

선형 및 커널 기반 모델

Lasso Regression: L1 규제를 통한 변수 선택 및 과적합 방지

ElasticNet: L1+L2 규제로 안정적인 모델링

Kernel Ridge Regression: RBF 커널 기반 비선형 패턴 학습

```
lasso = Lasso(alpha=0.0003, random_state=42)
ENet = ElasticNet(alpha=0.0005, l1_ratio=0.9)
KRR = KernelRidge(alpha=0.6, kernel='rbf')
```

4-1. 회귀 모델 정의

부스팅 모델

XGBoost: 규제가 강하고 성능이 우수한 대표적 부스팅 모델

LightGBM: Leaf-wise 성장 전략으로 빠르고 효율적인 학습

HistGradientBoosting: 히스토그램 기반의 빠른 Gradient Boosting 모델

```
model_xgb = XGBRegressor(  
    n_estimators=2000,  
    learning_rate=0.03,  
    max_depth=7,  
    subsample=0.8,  
    colsample_bytree=0.8  
)  
  
model_lgb = lgb.LGBMRegressor(  
    objective='regression',  
    num_leaves=31,  
    learning_rate=0.03,  
    n_estimators=3000,  
    max_bin=255,  
    bagging_fraction=0.8,  
    bagging_freq=5,  
    feature_fraction=0.8,  
    min_data_in_leaf=20,  
    min_sum_hessian_in_leaf=5,  
    random_state=42  
)  
  
model_hgb = HistGradientBoostingRegressor(max_depth=6, max_iter=600)
```

4-1. 회귀 모델 정의

Extra Trees Regressor

무작위성을 극대화한 앙상블 트리 모델
빠르고 안정적, 다양한 분할 방식으로 예측 다양성 증가

```
model_et = ExtraTreesRegressor(n_estimators=500, max_depth=None, random_state=42)
```

4-2. 스택킹

◆ Averaged Stacking 개요

- 여러 Base Model의 예측값을 교차검증 기반 OOF(Out-of-Fold) 예측으로 생성
- 생성된 OOF 예측값을 메타 모델의 학습 데이터로 사용
- 예측 단계에서는 Base Model들의 예측을 평균하여 메타 모델에 입력
- 모델 간 강점을 결합하여 예측 성능 향상 및 과적합 감소

```
class StackingAveragedModels:
    def __init__(self, base_models, meta_model, n_folds=5):
        self.base_models = base_models
        self.meta_model = meta_model
        self.n_folds = n_folds
```


4-2. 스택킹

◆ 코드 작동 방식

① fit() 단계

- 각 Base Model에 대해 K-Fold 교차검증 수행
- Fold별로 훈련된 Base Model들이 OOF 예측값을 생성
- 모든 Base Model의 OOF 예측값을 모아 메타 모델의 입력 특징(meta-features) 구성
- 메타 모델을 이 meta-features로 학습시켜 최종 스택킹 모델 완성

```
def fit(self, X, y):
    self.base_models_ = [list() for x in self.base_models]
    self.meta_model_ = clone(self.meta_model)
    kfold = KFold(n_splits=self.n_folds, shuffle=True, random_state=42)

    out_of_fold_predictions = np.zeros((X.shape[0], len(self.base_models)))

    for i, model in enumerate(self.base_models):
        for train_idx, val_idx in kfold.split(X, y):
            instance = clone(model)
            self.base_models_[i].append(instance)
            instance.fit(X[train_idx], y[train_idx])
            y_pred = instance.predict(X[val_idx])
            out_of_fold_predictions[val_idx, i] = y_pred

    self.meta_model_.fit(out_of_fold_predictions, y)
    return self
```

4-2. 스택킹

◆ 코드 작동 방식

② predict() 단계

- 각 Base Model이 전체 Train으로 학습된 여러 인스턴스로 Test 예측 수행
- Base Model들의 예측값을 ****평균(mean)****하여 안정성을 증가
- 평균된 예측값을 메타 모델에 입력하여 **최종 예측 생성**

```
def predict(self, X):  
    meta_features = np.column_stack([  
        np.column_stack([model.predict(X) for model in models]).mean(axis=1)  
        for models in self.base_models_  
    ])  
    return self.meta_model_.predict(meta_features)
```

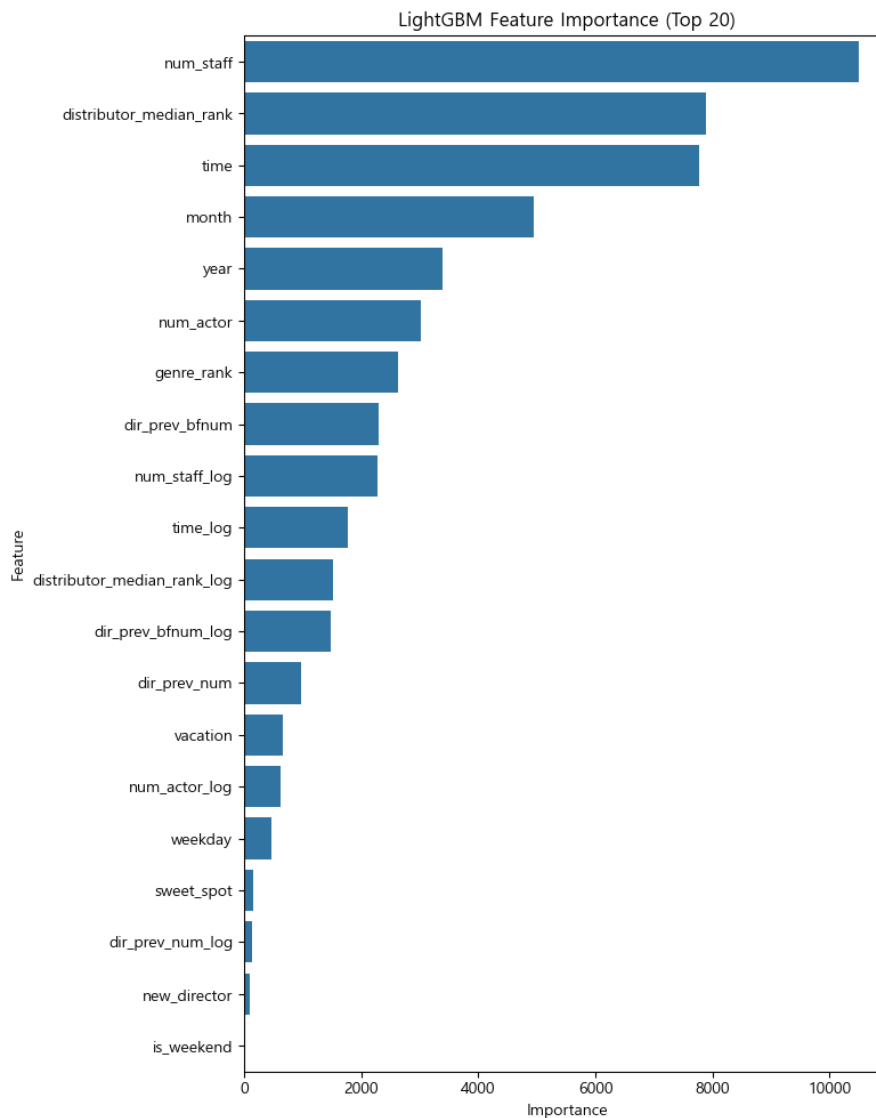
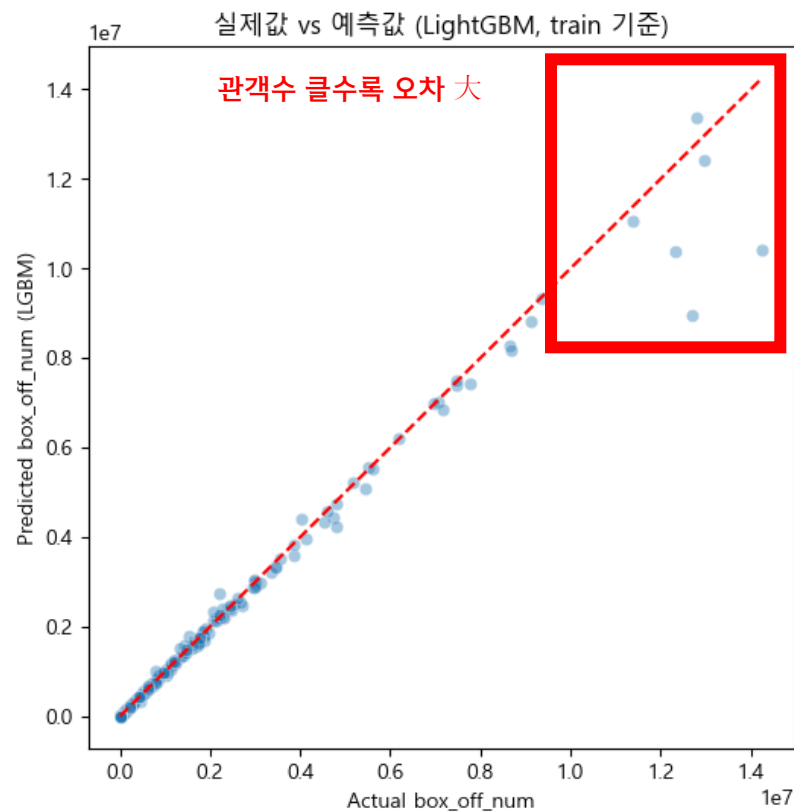
5. 모델 학습 및 예측

1) 개별 모델 학습 및 예측

```
stacked_averaged_models = StackingAveragedModels(  
    base_models=(ENet, KRR, lasso),  
    meta_model=lasso)  
  
# 각 모델 log 예측  
stacked_averaged_models.fit(X_scaled, train_y_log)  
stacked_log = stacked_averaged_models.predict(test_scaled)  
  
model_xgb.fit(X_scaled, train_y_log)  
xgb_log = model_xgb.predict(test_scaled)  
  
model_lgb.fit(X_scaled, train_y_log)  
lgb_log = model_lgb.predict(test_scaled)  
  
model_hgb.fit(X_scaled, train_y_log)  
hgb_log = model_hgb.predict(test_scaled)  
  
model_et.fit(X_scaled, train_y_log)  
et_log = model_et.predict(test_scaled)
```

5. 모델 학습 및 예측

1) 개별 모델 학습 및 예측



5. 모델 학습 및 예측

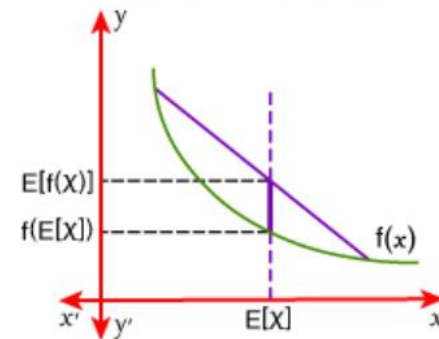
2) 앙상블

```
ensemble_log = (stacked_log + xgb_log + lgb_log + hgb_log + et_log) / 5  
final_pred = np.exp1(ensemble_log)
```

- 다른 특성을 가진 다양한 모델을 평균할수록 좋은 점수가 나왔음
- 성능 좋은 모델의 가중치를 높이면 성능 떨어짐
 - 해당 데이터는 개별 모델 성능보다 예측 분산을 줄이는 것이 더 중요
- 로그상태로 평균한 뒤 exp를 취할 때 성능이 더 좋음
 - $\exp(\text{mean}(\log)) < \text{mean}(\exp(\log))$
 - 과대예측 감소

For Convex Function

$$E[f(X)] \geq f(E[X])$$



5. 모델 학습 및 예측

3) 제출 데이터 생성

```
submission['box_off_num'] = final_pred
submission.to_csv("bestmodel_PATCH.csv", index=False)
print("Saved bestmodel_PATCH.csv")
```

➤ 제출점수: 1075432.99453573

감사합니다